

Warm up

What are the values of `x`, `y` and `z` after executing the following snippet?

```
int x = 0;  
int y = 10;  
int z = ((5 * !(x = 8)) && (y = 3)) * 10;
```

Warm up

What are the values of `x`, `y` and `z` after executing the following snippet?

```
int x = 0;  
int y = 10;  
int z = ((5 * !(x = 8)) && (y = 3)) * 10;
```

Answer: `x = 8`, `y = 3`, `z = 10`

Please don't write code like this... at least for this class

Instead, consider the [The International Obfuscated C Code Contest](#)

Precept 4: Writing and Building More Complex C Programs

Polly Ren

COS 217: Introduction to Programming Systems
Spring 2026
Princeton University

February 5, 2026

Overview

1 Programming in C

2 Multi-file programs

3 Summary

Primitive data types

Type	Description	Size (bytes)	Format
int or signed int	A (positive or negative) integer	4 (not always)	%d
char	An integer with smaller range than int. Usually represents a character according to a character code (e.g., ASCII) ¹	1	%c
float	A (positive or negative) <i>single</i> -precision floating point number	4 (not always)	scanf: %f printf: %f, %e, %E, %g
double	A (positive or negative) <i>double</i> -precision floating point number	8 (not always)	scanf: %lf printf: %lf, %e, %E, %g

¹Signedness varies by system (unsigned on armlab)

More primitive data types

- unsigned, signed, long and short are type modifiers that can be applied to primitive data types to change their range
- signed types can represent both positive and negative values
 - signed char, signed int
- unsigned types can only represent non-negative values, effectively doubling the upper range
 - unsigned char, unsigned int
- long types usually have a larger range than their non-long counterparts
 - long int (equivalently, long), long double, long long int
- short types usually have a smaller range than their non-short counterparts
 - short int
- Can be combined
 - unsigned long int, signed long int
 - But cannot contradict: unsigned signed int or unsigned double
- See *C Primitive Data Types* handout for more details and examples

C vs. Java: data types

Feature	C	Java
Unique Types	unsigned modifiers $\text{int} \in [-2^{31}, 2^{31} - 1]$ $\text{unsigned int} \in [0, 2^{32} - 1]$	boolean, byte
Size of char	1 byte (ASCII)	2 bytes (Unicode)
Type Sizes	System-dependent (except char)	Strictly specified and portable

The sizeof operator

- The C standard does not specify sizes of most data types (save for one-byte chars)
- sizeof returns the size (in bytes) of a data type or variable
 - Syntax: `sizeof(type)` or `sizeof(variable)`
 - Result type: `size_t` (unsigned integer, specific for representing sizes)
- Evaluated during the compilation stage of the build process

On armlab:

```
printf("Bytes per int:  %d\n", (int)sizeof(int));
```

```
Bytes per int:  4
```

Wait, what about strings?

- String literals are enclosed in double quotes, e.g., "hello"
 - Stored in read-only memory → immutable
- But, C has no built-in string data type
- Strings are instead represented as arrays of `chars`, with a sentinel value
 - End of the string indicated by a special NUL character (`'\0'`)
 - Example: the string literal "hello" is represented in memory as:
`['h', 'e', 'l', 'l', 'o', '\0']`
 - More on this after we cover arrays and pointers!

Operators and expressions

- See *C Operators* handout
- Notably:
 - Each operator has a precedence, lower number → higher precedence
 - Operators with same precedence evaluated based on associativity
 - Some operators differ between C and Java:
 - ◇ Java only: `>>>`, `new`, `instanceof`
 - ◇ C only: pointer-related operators (`*`, `&`, `[]`), `sizeof`
 - Relational and logical operators:
 - ◇ Java: relational (e.g., `<`) and logical (e.g., `&&`) operators evaluate to booleans
 - ◇ C: relational and logical operators evaluate to `int` (`false = 0`; `true = 1`)
 - String concatenation:
 - ◇ Java: can use `+` or `+=` to concatenate strings
 - ◇ C: cannot use `+` or `+=` to concatenate strings
 - Type demotions:
 - ◇ Java: demotions are not automatic
 - ◇ C: demotions are automatic in expressions with mixed types

Statements

- See *C Statements* handout
- C statements are (largely) similar to Java statements
- Must declare all variables before using them

For example:

```
int main() {  
    int x = 5;           /* declaration + initialisation */  
    int y;               /* declaration */  
    if (x == 1) {        /* if statement */  
        y = x + 5;       /* assignment */  
    } else {  
        y = x - 5;  
    }  
    return 0;           /* return statement */  
}
```

y is 0 at the end of the program, because x is not equal to 1

A common pitfall

- But, remember: no built-in boolean type in C $\rightarrow$ booleans are ints
 - Zero is false; *anything non-zero* is true

```
int main() {  
    int x = 5;  
    int y;  
    if (x = 1) {          /* only one equal */  
        y = x + 5;  
    } else {  
        y = x - 5;  
    }  
    return 0;  
}
```

- The assignment `x = 1` evaluates to 1 (true) $\rightarrow$ executes the `if` branch
- `y` is 10 at the end of the program(!)
- -Wall compiler option can flag these errors

Overview

1 Programming in C

2 Multi-file programs

3 Summary

Recall: the four-step build process

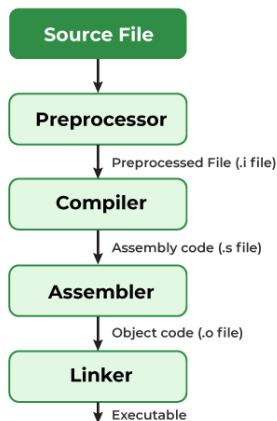


Image source:
<https://www.geeksforgeeks.org/c/compiling-a-c-program-behind-the-scenes/>

- Preprocessing
 - Removes comments and handles preprocessor directives
 - `gcc217 -E hello.c > hello.i`
- Compiling
 - Translates preprocessed C code into assembly code
 - `gcc217 -S hello.i`
- Assembling
 - Translates assembly code into machine code (object file)
 - `gcc217 -c hello.s`
- Linking
 - Combines object files and libraries into an executable
 - `gcc217 hello.o -o hello`
- Combined: `gcc217 hello.c -o hello`

Multi-file programs

- Large programs are often split across multiple source files
- Each source file is compiled into an object file separately
- The object files are then linked together to create the final executable
- Advantages:
 - Easier to manage and organise code
 - Only need to recompile (preprocess, compile, assemble) changed files
 - Facilitates code reuse

Writing multi-file programs

Walkthrough of `intmath` program, but here are the main points:

- static functions have internal linkage (only use in same `.c` file)
 - Similar to `private` in Java
- *Forward declarations* allow functions to be used before definition
 - Provide function signature (return type, name, parameter types, parameter names optional)
 - Why: compiler needs to see function signature (at minimum) before function call
- Header files (`.h`) provide the “interface” for a module
 - Function headers (declarations) go here
 - Implementation (definitions) should be in `.c` files
 - `#include` directive to include header files
 - ◇ Use quotes (`"intmath.h"`) for user-defined headers
 - ◇ Use angle brackets (`<stdio.h>`) for system headers

Building multi-file programs

- In a single command: `gcc217 intmath.c testintmath.c -o intmath`
 - Must specify as command-line arguments each `.c` file that comprises the program
 - Must not specify as command-line arguments any `.h` file, these are handled by preprocessor through `#include`
- More next week!

Preventing multiple inclusions

- Header files can be included in multiple source files
 - At best: inefficient because redundant declarations
 - At worst: compile-time errors due to multiple definitions
- To prevent multiple inclusions of the same header, use include guards

intmath.h:

```
#ifndef INTMATH_H      /* checks if INTMATH_H is not defined */
#define INTMATH_H      /* defines INTMATH_H */
...
#endif                /* marks the end of the conditional */
```

- First time the header file is included, INTMATH_H is not defined, so the contents of the file are included and INTMATH_H is defined
- Subsequent inclusions of the header file will find that INTMATH_H is already defined, so the body of the file is skipped

Overview

1 Programming in C

2 Multi-file programs

3 Summary

Summary

- Data types, operators and expressions, statements
- How C data types differ from Java
- Multi-file programs
- Dates to remember:
 - Assignment 1: due Tue, 2/10 at 9pm
 - Assignment 1 Quiz: Wed, 2/11 in lecture